Studio Nx

JS 294 — 이펙트 체인 관리

2026-10-19

JavaScript 294 — 이펙트 체인 관리 (`audio/effect_rack.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/effect_rack.js`
작업	S1-11a 재생 오디오 그래프: 트랙의 이펙트 목록을 읽어 체인을 만들고, 바뀌면 다시 잇고, 노브 값을 반영 (FR-243, FR-263)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 293 이펙트, 다음 295 레벨 미터

1. 하는 일

화면이 주는 "트랙의 이펙트 목록"(슬롯 순서, 바이패스, 파라미터)을 실제 이펙트 노드 체인으로 맞춥니다. 화면은 목록만 주고, 소리 쪽 변경은 모두 여기서 일어납니다.

```
getEffects(트랙) ─▶ 바이패스와 모르는 종류를 거름 ─▶ slot_index 순서로 정렬
        │
        ├ 새 이펙트          createEffect 로 만듭니다
        ├ 값이 바뀐 이펙트    update 로 소리에 반영합니다 (노드를 다시 만들지 않음)
        ├ 순서나 구성이 바뀜  믹서의 체인을 끊고 다시 잇습니다
        └ 사라진 이펙트       dispose 로 정리합니다
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession.start`	재생을 시작할 때 `sync`를 불러 현재 이펙트 구성을 소리에 만듭니다.
`PlaybackSession.refreshMixer`	재생 중 150ms마다 `sync`를 불러 이펙트 추가, 삭제, 순서, 바이패스, 노브 변경을 소리에 반영합니다.
`PlaybackSession.stop`	`flush`로 이펙트를 비워서 딜레이와 리버브의 꼬리가 다음 재생에 섞이지 않게 합니다.

3. 코드

```
_syncTrack(key, trackId) {
  const items = (this.getEffects(trackId) || [])
    .filter(function (e) { return !e.is_bypassed && EFFECT_CODES.indexOf(e.plugin_code) >= 0; })
    .sort(function (a, b) { return a.slot_index - b.slot_index; });

  const self = this;
  const chain = [];
  items.forEach(function (e) {
    const params = resolveParams(e.schema, e.params);
    const sig = JSON.stringify(params);
    let entry = self.live[e.id];
    if (entry && entry.code !== e.plugin_code) { entry.fx.dispose(); entry = null; delete self.live[e.i
d]; }
    if (!entry) {
```

```
      entry = { fx: createEffect(self.context, e.plugin_code, params, self.getBpm), code: e.plugin_code,
sig: sig, track: key };
      self.live[e.id] = entry;
    } else if (entry.sig !== sig) {
      entry.fx.update(params);
      entry.sig = sig;
    } else if (e.plugin_code === 'delay') {
      entry.fx.update(params);    // 템포 동기 딜레이는 BPM 이 바뀌면 값이 달라져서 매번 다시 계산함
    }
    chain.push({ id: e.id, fx: entry.fx });
  });

  const signature = chain.map(function (c) { return c.id; }).join(',');
  if (this.chains[key] !== signature) {
    const list = chain.map(function (c) { return c.fx; });
    if (key === 'master') this.mixer.setMasterEffects(list);
    else this.mixer.setTrackEffects(trackId, list);
    this.chains[key] = signature;
  }

  // 목록에서 사라진 이펙트(삭제 · 종류 바뀜)는 정리
  const keep = {};
  chain.forEach(function (c) { keep[c.id] = true; });
  Object.keys(this.live).forEach(function (id) {
    if (self.live[id].track === key && !keep[id]) { self.live[id].fx.dispose(); delete self.live[id]; }
  });
}
```

```
sync(tracks) {
  const self = this;
  (tracks || []).forEach(function (t) {
    self._syncTrack(t.track_type === 'MASTER' ? 'master' : String(t.track_id), t.track_id);
  });
}
```

```
flush() {
  const self = this;
  Object.keys(this.chains).forEach(function (key) {
    if (key === 'master') self.mixer.setMasterEffects([]);
    else self.mixer.setTrackEffects(key, []);
  });
  Object.keys(this.live).forEach(function (id) { self.live[id].fx.dispose(); });
  this.live = {};
  this.chains = {};
}
```

4. 최신 문법으로 쓰면

```
// 체인 구성이 바뀌었는지 비교하는 부분을 최신 문법으로 쓴 모양
const signature = chain.map(({ id }) => id).join(',');
if (this.chains[key] !== signature) {
  const list = chain.map(({ fx }) => fx);
  key === 'master' ? this.mixer.setMasterEffects(list) : this.mixer.setTrackEffects(trackId, list);
```

```
  this.chains[key] = signature;
}
```

전통 문법 (실제 코드)	최신 문법
`function (c) { return c.id; }`	`({ id }) => id`
`if (key === 'master') ... else ...`	삼항 연산자
`Object.keys(obj).forEach(function (id) { ... })`	`for (const id of Object.keys(obj))`

5. 왜 이렇게

선택	이유
이펙트 번호를 키로 노드를 보관합니다 (`live`)	같은 이펙트의 노브만 바뀌었을 때 노드를 다시 만들지 않고 값만 바꿉니다. 재생이 끊기지 않습니다.
파라미터를 문자열로 만들어 비교합니다 (`sig`)	값이 안 바뀌었으면 아무것도 하지 않습니다. 150ms마다 불러도 가볍습니다.
체인 구성이 바뀔 때만 다시 잇습니다	연결을 끊고 잇는 순간 소리가 잠깐 끊길 수 있어서, 이펙트 목록이 실제로 달라졌을 때만 합니다.
딜레이는 매번 `update`를 부릅니다	템포 동기 딜레이는 BPM이 바뀌면 시간이 달라지는데, 파라미터 값은 그대로라 비교로는 알 수 없습니다.
바이패스한 이펙트는 체인에서 빼고 노드도 정리합니다	꺼진 이펙트가 소리에 영향을 주지 않게 하는 가장 단순한 방법입니다. 다시 켜면 새로 만들어서 같은 값으로 소리가 나지만, 이전의 딜레이와 리버브 꼬리는 이어지지 않습니다.
MASTER 트랙은 특별한 키 (`master`)를 씁니다	마스터 이펙트는 마스터 볼륨 뒤에 들어가서 일반 트랙과 다른 연결을 씁니다.
정지에서 `flush`합니다	이펙트를 비우고 다음 재생에서 새로 만듭니다. 이전 재생의 꼬리가 섞이는 것을 막습니다.
모르는 종류(Sampler 등)는 건너뜁니다	`EFFECT_CODES`에 없는 코드는 소리를 바꾸지 않고 지나갑니다.

6. 시험

- `docs/handoff/s1-11a_tests/ui_fx.mjs` (8건)는 실제 화면에서 다음을 확인합니다.

시험	결과
이펙트 없음	피크가 사인파 진폭 0.4와 같습니다.
트랙에 Gain -12dB	피크가 약 1/4(0.1)입니다.
바이패스	원래 소리(0.4)로 돌아옵니다.
바이패스를 풀고 Gain -24dB	피크가 약 1/16(0.025)입니다.
마스터 트랙에 Limiter 천장 -12dB	최종 출력의 피크가 0.3 아래로 줄어듭니다.

- 시험 데이터는 이펙트가 없는 시험 전용 오디오 트랙을 새로 만들어 쓰고, 끝나면 모두 지웁니다 (기존 트랙에는 이미 이펙트가 있어서 기준 소리가 달랐기 때문입니다).